Counting the Cold: Team Build Plan

Last updated Oct 23, 2026 · Chuckisawsome12345/counting-the-cold

Three people have 24 hours at Reboot Earth (UC Davis, Oct 23–24 2026) to ship a free web app that gives any California orchard its chill and GDD, checked against CIMIS stations. One-liner: "*Your orchard's chill and heat, not a station's.*"

How to use this plan

This file is the single source of truth for the plan. It replaced the Claude Doc we started in, because everyone's Claude can read and edit a repo file.

- **Teammates:** read Onboarding and Roles first, pick a role (A, B, or C), then take the GitHub issues labeled for your role.
- **AI assistants (Claude Code etc.):** read "Rules for AI assistants" below before touching the repo. Your human tells you their role. The root `CLAUDE.md` repeats the essentials.
- **Changing the plan:** edit this file in a small PR labeled `plan`, and @mention the other two so they see it. Changes to the API contract also need a `contract` PR (see Roles).
- **Progress:** tracked on the GitHub milestones and issues, not in this file.
- **Printable copy:** `docs/BUILD_PLAN.pdf` is a snapshot of this file. Regenerate it after big changes.

Context

Growers track chill and GDD from CIMIS stations that are often far away, in a different microclimate, or closed; we give any pin on a map its own numbers. The challenge is "Counting the Cold" from UC Davis FNRIC.

- **Chill:** winter cold accumulated from hourly temperatures. It predicts bud break.
- **Growing degree days (GDD):** heat accumulated from daily max and min. It predicts pests, disease, and harvest.

A grower drops a pin on their orchard and sees:

1. Chill and GDD for the season so far
2. A comparison to the last 10 seasons
3. A 7–14 day projection

Differentiator: we validate gridded weather data against CIMIS stations, show the error, and bias-correct orchard estimates. The demo shows a Butte County orchard near the closed Durham station still getting numbers.

Facts verified Oct 23 2026:

- **The CIMIS API is flaky.** `/api/station` returned `E90002: Unhandled server exception` for part of the evening, then recovered. We hardcode station metadata (`backend/data/stations.json`) and pull CIMIS data early into committed files, so the running app never calls CIMIS.
- **Durham #12 disconnected on 2025-07-30**, with records back to 1982. The nearest active stations are **Biggs #244** (25 km, since 2015), **Williams #250** (53 km, since 2016), and **Gerber South #222** (56 km, since 2014). Orland, Colusa, and Nicolaus are all closed.
- **Open-Meteo's archive API works.** It returns hourly °F for any lat/lon with `timezone=GMT`.
- **FNRIC counting windows:** Dynamic Model chill portions count Sep 1 through Aug 31. Chill hours count Nov 1 through the end of February.

- **FNRIC's almond hull split model uses GDD90:** degree days from full bloom to 90 days after, single-sine method (Tombesi et al. 2010). It does not use degree hours.

Team decisions

These are settled unless the team agrees to change them here.

Topic	Decision
Repo	Public GitHub repo. Main is protected, so every change goes through a PR.
Plan	This file. Changes go through a PR labeled `plan`.
Team	3 roles (A, B, C), each owning separate folders. People self-assign issues by role label.
Seasons	A season runs Sep 1–Aug 31 (label "2025-26"). Portions start Sep 1, chill hours and Utah start Nov 1, GDD starts Jan 1 or at biofix. All start dates are configurable. The dashboard opens on 2025-26, with a toggle to 2026-27.
Units	°F everywhere a user sees it. The Dynamic Model converts to Kelvin internally.
Time	The internal hourly index is fixed PST (UTC−8, no daylight saving) to match CIMIS. Open-Meteo is fetched in GMT and shifted. CIMIS hour `2400` becomes 00:00 the next day.

Onboarding

Each teammate needs about 10 minutes. The full checklist is issue #1.

1. Accept the GitHub collaborator invite and clone the repo.
2. Find your role in the Roles table below: **A (Data & API)** is @Chuckisawsome12345, **B (Science & Validation)** is @karthikmaheshwaran, **C (Frontend & Demo)** is @Zooomer69.
3. Create your env file: `cp .env.example .env`.
   - Only B (who runs the CIMIS fetcher) and A (who deploys the backend) need `CIMIS_KEY`. Get a free key at et.water.ca.gov.
   - Never commit the key, paste it in chat, or put it in an issue.
4. Start the backend: `cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt && uvicorn app.main:app --reload`
5. Start the frontend: `cd frontend && npm install && npm run dev`. It proxies `/api` to `localhost:8000`.
6. Your role's issues are already assigned to you: `gh issue list --assignee @me`.

Milestones

The app works end to end by H4 (H = hours from kickoff), and features freeze at H20. Live status is on each milestone's GitHub page.

When	Milestone	Done means
H0–H1	M0 Setup	Repo, scaffold, contract, mock JSON, CI, and issues exist. Everyone runs both apps locally.
H1–H4	M1 Walking skeleton	A dropped pin returns real Open-Meteo temps, with chill hours and GDD in cards. Deployed to Render and Vercel.

When	Milestone	Done means
H4–H8	M2 Core science	Utah and Dynamic Model with golden tests, gap filling, cache, 10-season history, forecast merge. Season chart and settings bar.
H8–H13	M3 Differentiator	CIMIS pulled for all 9 stations, `validation.json` built, correction wired in. Validation tab, station markers, Durham preset.
H13–H17	M4 Projection and polish	p10/p50/p90 projection, prewarm and snapshots with `DEMO_MODE`, error and stale handling, responsive layout.
H17–H20	M5 Stretch	Standout features in priority order (accuracy map, statewide chill map, ML correction, % of chill requirement met), then pin vs station compare, hull split, public `/api/v1` docs. Start only after M4 is done.
H20–H24	M6 Freeze and demo	Feature freeze at H20. Final deploy and snapshot at H21. Rehearse 3 times, record a backup video, write Devpost and README.

Suggested sleep shifts, about 3 hours each so someone can always merge: C sleeps H11–H14, B H14–H17, A H17–H20.

Roles and conflict prevention

Each role owns separate folders, so no two people edit the same file. Roles were assigned on Oct 23, and `.github/CODEOWNERS` maps the same folders to the same people, so GitHub requests the owner's review on any PR that touches their files. The full file-by-file table is in `docs/CONTRIBUTING.md`.

Role	Person	Owns (only this person edits)	Core work	Issues
A: Data & API (strongest backend person)	@Chuckisawsome12345	`backend/app/{main,config,schemas}.py`, `routes/`, `sources/` (except `cimis.py`), `pipeline/`, `scripts/prewarm.py`, `scripts/make_mock.py`, `data/stations.json`, `data/snapshots/`, `requirements.txt`, root files, Render deploy	Open-Meteo client and merge, PST normalization, SQLite cache, `/api/point`, history and forecast assembly, correction wiring, stations and validation routes, snapshots and DEMO_MODE, deploy, README	#2–#12
B: Science & Validation (strongest Python and math person)	@karthikmaheshwaran	`backend/app/calc/`, `backend/app/validation/`, `backend/tests/` (except test_api and test_sources), `sources/cimis.py`, `scripts/fetch_cimis.py`, `scripts/build_validation.py`, `data/cimis/`, `data/validation.json`, test fixtures	FNRIC reference values; chill hours, Utah, GDD, and seasons by H2 (this unblocks A); Dynamic Model with golden tests; gap filling; CIMIS fetcher (start early); bias model; leave-one-out validation; projection; hull split	#13–#21

Role	Person	Owns (only this person edits)	Core work	Issues
C: Frontend & Demo	@Zooomer69	`frontend/**`, `docs/demo-script.md`, Devpost text, slides, Vercel deploy	Map and pins, cards, season chart, settings, quality badge, validation tab, correction info, compare panel, demo script, backup video	#22–#31

Rules that prevent overwriting each other's work:

1. **Scaffold first.** One commit created every shared file before anyone started: root config, CI, folder skeletons, stub modules with final function signatures, `api-contract.md`, and the mock JSON. (Done.)
2. **Edit only your own folders.** If you need something from another area, comment on the owner's issue or ping them.
3. **Contract changes** go in a PR labeled `contract` that edits `docs/api-contract.md` and @mentions the other two. A mirrors the change in `schemas.py` (and re-runs `scripts/make_mock.py`), C in `types.ts`.
4. **Plan changes** go in a small PR labeled `plan` that edits this file and @mentions the other two.
5. **Lockfiles:** only C touches `package-lock.json`, only A touches `requirements.txt`. B's dependencies were added at scaffold time.
6. **One branch per issue,** prefixed by role: `data/4-point-v1`, `sci/16-dynamic-model`, `ui/25-season-chart`. PRs say `Closes #N`. Run `git pull --rebase origin main` before opening one. Keep PRs small and merge every 1–2 hours.
7. **GitHub enforces it.** A ruleset on `main` requires a PR and passing `backend-tests` and `frontend-build` checks, and blocks force pushes and deletion. Approvals required: 0, for speed. Squash merge only, and branches auto-delete after merge.
8. **Secrets:** `.env` is gitignored. `CIMIS_KEY` lives only in local `.env` files and in Render's environment, and is never sent to the frontend.

Tech stack

We use FastAPI (Python) with React/Vite (TypeScript), Leaflet, and Recharts. Streamlit would be faster to start, but its map pins and chart interactions look clunky in a demo. The split stack also lets 3 people work in parallel without touching each other's files.

- **Backend:** Python 3.12+, FastAPI, uvicorn, httpx, pandas, numpy, pydantic v2, python-dotenv, pytest (versions pinned in `backend/requirements.txt`). The cache is a plain SQLite file through the `sqlite3` module, with no ORM.
- **Frontend:** Vite react-ts template, react-leaflet with OpenStreetMap tiles, Recharts, plain CSS (or Tailwind if C prefers). TypeScript runs with `strict: false`; it's there so `types.ts` acts as the contract with the backend.
- **Infra:** GitHub Actions CI (pytest and `npm run build`), Render for the backend, Vercel for the frontend.

Folder structure

The letter in parentheses is the role that owns each path.

```
counting-the-cold/
├── README.md  CLAUDE.md          # setup + run; rules for AI assistants (A)
├── .gitignore  .env.example      # CIMIS_KEY=, DEMO_MODE=0, ALLOWED_ORIGINS= (A)
├── .claude/launch.json           # dev servers for the Claude desktop app (A)
├── .github/
│   ├── workflows/ci.yml          # backend-tests + frontend-build, always run
│   └── pull_request_template.md
├── docs/
│   ├── BUILD_PLAN.md  .pdf       # this plan (shared, `plan` PRs) + printable copy
│   ├── CONTRIBUTING.md           # git workflow + file-by-file ownership
│   ├── api-contract.md           # source of truth for JSON shapes (shared, `contract` PRs)
│   └── demo-script.md            # (C)
├── backend/
│   ├── requirements.txt  pytest.ini                        # (A)
│   ├── app/
│   │   ├── main.py config.py schemas.py                    # (A)
│   │   ├── routes/     point.py stations.py validation.py meta.py   # (A)
│   │   ├── sources/    openmeteo.py cache.py snapshots.py  # (A); cimis.py (B)
│   │   ├── pipeline/   build_point.py timeseries.py        # (A)
│   │   ├── calc/       chill.py gdd.py seasons.py gaps.py
│   │   │               projection.py hullsplit.py crops.py # (B)
│   │   └── validation/ bias.py metrics.py                  # (B)
│   ├── scripts/
│   │   ├── fetch_cimis.py        # (B) resumable download -> data/cimis/
│   │   ├── build_validation.py   # (B) -> data/validation.json
│   │   ├── prewarm.py            # (A) demo pins -> cache/snapshots
│   │   └── make_mock.py          # (A) synthetic /api/point responses
│   ├── data/
│   │   ├── stations.json         # 9 stations, Durham active=false (A)
│   │   ├── mock_point*.json      # synthetic responses served until the pipeline lands (A)
│   │   ├── cimis/                # station-season JSON.gz (B)
│   │   ├── snapshots/            # demo-pin responses for DEMO_MODE (A)
│   │   └── validation.json       # (B)
│   └── tests/                    # calc + validation tests, helpers.py (B); test_api, test_sources (A)
│       └── fixtures/             # fnric_reference.json, cimis_sample.json (B)
└── frontend/                     # everything here is (C)
    ├── vite.config.ts            # proxy /api -> localhost:8000
    └── src/
        ├── api/        client.ts types.ts mock/point.json mock/point_current.json
        ├── hooks/      usePoint.ts
        └── components/ MapPicker SettingsBar SummaryCards SeasonChart
                        ForecastChart ValidationPanel DataQualityBadge
                        CorrectionInfo StationCompare HullSplitCard
```

Data pipeline and caching

Every pin is served from Open-Meteo through a three-layer cache. CIMIS is used only offline, for validation. A orchestrates `/api/point` and calls B's pure functions.

Request flow for `/api/point`:

1. **Snap** lat/lon to 0.1° (the ERA5-Land grid) so nearby pins share cache entries. The UI still shows the exact pin.
2. **Fetch** hourly `temperature_2m` in °F, GMT:
   - Archive API (`archive-api.open-meteo.com/v1/archive`) from Sep 1 ten seasons back through today – 6 days. The archive lags about 5 days.
   - Forecast API (`api.open-meteo.com/v1/forecast`) with `past_days=10&forecast_days=16`.
3. **Merge:** archive wins where both overlap, the forecast fills recent days and the future. Each row gets a `source` tag.

4. **Normalize:** convert to PST, run `calc.gaps.fill_gaps`, apply the bias correction, then derive daily tmax and tmin from hourly data in PST days.
5. **Compute** the models per season, plus history, forecast, and climatology, and return the response.

CIMIS (offline only, `scripts/fetch_cimis.py`):

- **Stations:** 6 Davis, 70 Manteca, 71 Modesto, 39 Parlier, 5 Shafter; 12 Durham (history through 2025-07-30); and Durham's nearest active neighbors 244 Biggs, 250 Williams, 222 Gerber South.
- **Requests:** hourly `hly-air-tmp` and daily max/min, one station-season per request, falling back to monthly chunks if refused. Header `Ocp-Apim-Subscription-Key`; records at `Data > Providers > Records`.
- **Resilience:** retry with backoff and write each station-season to its own file, so a crash or outage resumes where it stopped.
- **QC flags:** blank is good, `Y` is kept but flagged, and `M`, `R`, `Q`, `S`, `N` count as missing.
- **Storage:** downloaded files are committed to `data/cimis/`.

Cache layers:

Layer	What	TTL
L1 SQLite (`backend/.cache/http.sqlite`, gitignored)	Raw upstream responses, gzipped, keyed by source and params	Archive and past CIMIS seasons: forever. Forecast: 1 h.
L2 in-memory LRU	Computed `/api/point` result per snapped cell and settings	15 min
L3 committed snapshots (`data/snapshots/*.json.gz`)	Full responses for demo pins and stations	When `DEMO_MODE=1`, or when a live fetch fails with no cache entry

Fallback chain: L2 → L1 fresh → live fetch → L1 stale (response says `stale: true`, UI shows a banner) → L3 snapshot → 503 with a clear message.

Rate budget: Open-Meteo's free tier allows about 600 calls/min, 5,000/hour, and 10,000/day, and counts long multi-week requests as several calls (confirm on their terms page). Ten years of hourly data for a new pin is expensive, so we snap, cache forever, and prewarm demo pins at startup.

Calculation module (B)

All chill and GDD math lives in pure functions with no I/O. Inputs are pandas Series indexed by PST timestamps, in °F, with NaN meaning missing. The stubs in `backend/app/calc/` already have these signatures, and tests marked `@stub` are waiting for them.

```
# calc/chill.py
chill_hours(t, lo=32.0, hi=45.0) -> Series   # 1 if lo <= t <= hi, else 0; NaN stays NaN
utah_units(t) -> Series                       # Richardson bands, °F:
    # <=34: 0 | 35-36: 0.5 | 37-48: 1 | 49-54: 0.5 | 55-60: 0 | 61-65: -0.5 | >65: -1
    # season cumulative floored at 0 (documented, toggleable)
dynamic_portions(t) -> Series                 # cumulative portions (Fishman/Erez)
    # e0=4153.5 e1=12888.8 a0=139500 a1=2.567e18 slp=1.6 tetmlt=277
    # chillR algorithm, Kelvin internally; missing hours carry state, skip increment

# calc/gdd.py
daily_gdd(tmax, tmin, base, upper=None, method="average" | "single_sine") -> Series
    # average = max(0, (tmax + tmin)/2 - base), horizontal cutoff at upper
    # single_sine (Zalom 1983) is used by hull split and UC IPM

# calc/crops.py      almond-NOW 55/94°F, walnut-codling-moth 50/88°F, generic 50, custom
# calc/seasons.py    season_window(season, start_mmdd) -> (start, end); handles Feb 29
# calc/gaps.py       fill_gaps(t, max_interp_h=3) -> (filled, GapReport)
# calc/projection.py forecast continuation + analog-season p10/p50/p90
# calc/hullsplit.py  (stretch) GDD90 from bloom date, Tombesi 2010 coefficients
```

Tests run offline with pytest in CI:

- **Hand cases:** chill hours for [31, 32, 40, 45, 46] °F give [0, 1, 1, 1, 0]; every Utah band boundary; GDD with tmax 80, tmin 50, base 50 gives 15, below base gives 0, upper cutoff applies; season windows in leap years.
- **Properties:** Dynamic Model portions never decrease, and a constant 80°F series gives about 0. Gaps of 3 h or less are interpolated; a 5 h gap appears in `long_gaps`.
- **Bias:** gridded = observed − 2°F gives a +2 table, and applying it returns the observed series.
- **Golden values:** in H0–H2, B records 4–6 published FNRIC "Cumulative Chilling Portions" values (Davis #6, Parlier #39, Shafter #5, recent seasons) in `tests/fixtures/fnric_reference.json`. Tests recompute them from committed CIMIS data within ±1 portion or ±2%. FNRIC fills gaps from a backup station, which explains small differences.
- **Optional:** if anyone has R, cross-check against `chillR::Dynamic_Model`.

API endpoints (A)

One endpoint, `/api/point`, carries everything the dashboard needs. FastAPI serves auto-generated docs at `/docs`. The full shapes live in `docs/api-contract.md`; this is the summary.

Method	Path	Returns
GET	`/api/health`	`{ok, cache_entries, demo_mode}`, used for warm-up pings
GET	`/api/point?lat&lon&season=2025&crop=almond&portions_start=09-01&hours_start=11-01&gdd_start=01-01&corrected=true`	Everything the dashboard needs
GET	`/api/stations`	`stations.json` plus each station's validation summary
GET	`/api/validation`	Raw vs corrected error per station, season, and metric, plus overall MAE
GET	`/api/crops`	Crop presets

Method	Path	Returns
GET	`/api/compare?lat&lon&season`	Stretch: pin vs nearest station, side by side
GET	`/api/hullsplit?lat&lon&bloom=YYYY-MM-DD&cultivar=`	Stretch
GET	`/api/v1/models?lat&lon`	Stretch: public open API with totals for every model
GET	`/api/grid?season&metric`	Stretch: precomputed statewide chill grid for the map layer

`/api/point` response, top level (see the contract for every field): `mock`, `location`, `season`, `settings`, `totals`, `daily[]` (cumulative season-to-date per day), `forecast` (null once a season is over), `history[]` (10 prior seasons), `climatology` (end-of-season p10/p50/p90, null once over), `quality`, `correction`, `nearest_station`. Until A's pipeline lands it returns synthetic data from `scripts/make_mock.py` with `"mock": true`.

Differentiator: validation and bias correction

We correct gridded temperatures with station offsets and prove it works with leave-one-station-out tests. B builds the math, and A wires it into the API.

- **Bias model:** for each station, a 12 × 24 table of mean (CIMIS – gridded) °F by month and hour of day, fit on hourly data.
- **Correcting an orchard pin:** blend station tables by inverse-distance weighting (power 2, stations within 150 km). If a closed station within 10 km has at least 3 seasons of history, weight its own table heavily. The response lists the stations used and their weights.
- **The Durham line:** "*The station closed, but its decade of records still calibrates our estimate.*"
- **Honest validation** (`build_validation.py` → `validation.json`): for each station and season, compute every metric from CIMIS (truth), raw gridded data, and corrected gridded data. Station X is corrected using only the other stations' tables, which is the situation a real orchard pin is in. Report MAE per metric, raw vs corrected. Durham through 2024-25 is the headline case.
- **Model check:** the script also scores Open-Meteo's `best_match`, `era5_land`, and `ecmwf_ifs` models, and we use whichever has the lowest error.

UI (C): one-page dashboard

A map on the left and the numbers on the right, with a second tab that shows how accurate we are. The map, demo pins, season dropdown, and summary cards already work against the mock; the rest are placeholders in `frontend/src/components/`.

- **Header:** name, one-liner, and two tabs: **Orchard** (default) and **How accurate is this?**
- **Map:** click to drop a pin. Active stations are filled markers; closed ones are hollow grey, with a "Durham, closed 2025-07-30" tooltip. Demo shortcut buttons: Butte orchard near Durham, Davis, Parlier. Stretch: address search via Nominatim.
- **Settings bar:** season dropdown, crop preset, the three start dates, and a bias-corrected toggle.
- **Summary cards:** Chill Portions, Chill Hours, Utah, GDD. Each shows the value, ▲/▼ vs the 10-season median, the +14-day projection, and the expected ± error.

- **Season chart (Recharts):** cumulative value by day of season. Past seasons faint grey, median dashed, current season bold, forecast dotted for 14 days, then a p10–p90 band to season end. Tabs switch between the four metrics.
- **Season totals bar chart** for the last 10 seasons.
- **Data quality:** badge with missing, interpolated, and long-gap hours, plus a stale-data banner.
- **Correction info:** e.g. "Corrected using Durham (historical) 97%, Biggs 2%, Williams 1%."
- **Validation tab:** per-station table and bars of raw vs corrected error, with a Durham callout.
- **Stretch:** station compare panel and hull split card.
- **States:** loading skeletons, error and stale banners.

Standout features (after M4)

These four additions make the demo land harder with judges, honestly: each one shows something real. Build them in this order once M4 is done; each is its own issue in milestone M5.

Priority	Feature	Why it matters	Who
1	**Accuracy map**	The validation tab becomes a map: each station is a bubble whose error shrinks from raw to corrected. Our proof of accuracy, visible at a glance.	C (data already in `/api/validation`)
2	**Statewide chill map**	A layer coloring the Central Valley by this season's chill, from a grid computed ahead of time. Says "every orchard, not 9 stations" in one picture.	A builds the grid and `/api/grid`; C draws the layer
3	**ML bias correction**	A machine-learning model competes with the simple 12 × 24 table on the same leave-one-station-out test. We use whichever wins and show both scores.	B (A adds the dependency)
4	**% of chill requirement met**	Pick a variety (e.g. Nonpareil almond) and see how close the orchard is to its chill requirement. Turns a number into a decision.	B (requirements + math), A (API field), C (card)

Details:

- **Accuracy map:** stations from `/api/stations`, errors from `/api/validation`. A toggle or animation goes from raw to corrected MAE per station; Durham gets a callout.
- **Statewide chill map:** `scripts/build_grid.py` (A) computes one season's corrected chill portions on a coarse grid (about 0.25°, Central Valley only, roughly 150 cells) and commits `data/grid/<season>.json`. It runs offline once, so it costs nothing at demo time and stays inside Open-Meteo's rate budget. `/api/grid` serves it; the shape goes in `docs/api-contract.md` first.
- **ML bias correction:** gradient-boosted trees (scikit-learn) predicting the hourly CIMIS – gridded offset from month, hour, gridded temperature, elevation, and distance to the coast. Same leave-one-station-out split as the table, so the comparison is fair. `validation.json` reports both MAEs. If ML doesn't beat the table, we say so; the honest comparison is itself a good demo moment.
- **% of chill requirement met:** chill-portion requirements per variety from published UC sources, cited in `calc/cultivars.py` (B). `/api/point` gains a requirement field through a `contract` PR (A); C adds a variety picker and a progress card. No requirement goes in without a source.

Risks and fallbacks

The demo survives any single upstream outage because demo pins are cached and snapshotted in the repo.

Risk	Fallback
CIMIS API down (it was, part of Oct 23) or refusing large requests	Hardcoded station metadata. Resumable fetcher retries and falls back to monthly chunks. Data committed once fetched. The app never calls CIMIS at runtime.
Open-Meteo down or rate-limited	Snapping and caching, prewarmed demo pins, stale cache with a banner, L3 snapshots, `DEMO_MODE=1` during judging
Archive lags about 5 days	Merge in the forecast API's `past_days`
Gridded data (about 9–11 km) misses orchard microclimates	Bias correction plus a visible expected error. PRISM 800 m can be added later as a validation layer.
Time zone or hour misalignment (CIMIS PST, hour `2400`)	One normalization function with a unit test
Missing or flagged data	Interpolate gaps of 3 h or less, flag longer ones in the UI with counts
Dynamic Model bug	Golden tests against FNRIC and chillR
Render free-tier cold start (about 50 s)	Ping `/api/health` 2 minutes before judging. Railway as backup.
Venue Wi-Fi fails	Run locally with `DEMO_MODE=1`, plus a backup video
Merge conflicts	Folder ownership, scaffold first, small PRs
Scope creep	Stretch goals only after M4. Hard freeze at H20.

Deployment

The backend goes on Render and the frontend on Vercel. Deploy the skeleton at H4, not at the end.

- **Backend on Render** (free web service, issue #5): root directory `backend`, build `pip install -r requirements.txt`, start `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Env vars: `PYTHON_VERSION=3.12.x`, `CIMIS_KEY`, `ALLOWED_ORIGINS=<vercel url>`, `DEMO_MODE`. The disk is ephemeral, so committed snapshots and a startup prewarm cover the demo.
- **Frontend on Vercel** (issue #24): root `frontend`, Vite preset, `VITE_API_URL=<render url>`. Every PR gets a preview URL.
- **Schedule:** pushes to main auto-deploy both. Final tagged deploy at H21.
- **Backups:** Railway for the backend, a laptop running `uvicorn` and `npm run preview`, and the recorded video.

Demo script (about 2:30)

The story is the closed Durham station: a grower loses their station, and we still give them numbers they can trust. C owns the working copy in `docs/demo-script.md`; fill in X and Y from `validation.json`.

1. **0:00 Hook.** "In July 2025, CIMIS station 12 in Durham, Butte County, shut down. Every almond grower who used it lost their chill count for last winter."
2. **0:20 Drop a pin** on a Butte orchard near Durham. The 2025-26 cards appear: portions, hours, Utah, GDD. "No station, no hardware. Your orchard's chill and heat, not a station's."

3. **0:50 History chart.** This season vs the last 10 and the median. "Chill finished X% above normal."
4. **1:10 Toggle to 2026-27.** Portions have been accumulating since Sep 1. The 14-day forecast runs into November with an end-of-season range.
5. **1:30 Trust: the validation tab.** "We checked ourselves against 9 CIMIS stations. Raw gridded data was off by X portions; corrected, by Y. And Durham's decade of records still calibrates the estimate after the station closed."
6. **2:00 Microclimate.** Pin vs nearest station side by side (stretch), or move the pin and watch the numbers change.
7. **2:15 Open and free.** Show `/docs`, the open API. Close on the one-liner.

Rules for AI assistants

Any Claude Code (or other assistant) working in this repo follows these rules. Your human tells you their role: A, B, or C. The root `CLAUDE.md` loads these automatically in Claude Code.

1. **Read first:** this file, `docs/CONTRIBUTING.md`, and `docs/api-contract.md`.
2. **Only create or edit files your role owns** (see Roles and `docs/CONTRIBUTING.md`). If a task needs a change elsewhere, stop and tell your human who owns it.
3. **Never change the shape of `docs/api-contract.md`, `backend/app/schemas.py`, or `frontend/src/api/types.ts` on your own.** Contract changes go through a `contract` PR.
4. **Plan edits** go through a small `plan` PR that changes only this file; don't rewrite sections that belong to another role without asking.
5. **Don't add dependencies** unless your role owns that lockfile or requirements file. Ask the owner.
6. **Git:** branch `<data|sci|ui>/<issue#>-<slug>`, rebase on `origin/main` before pushing, small PR with `Closes #N`. Never push to `main`, force-push a shared branch, or rewrite someone else's commits.
7. **Never read, print, commit, or log `CIMIS_KEY`** or the contents of `.env`.
8. **Stay offline-safe:** no CIMIS calls on the request path, every upstream call goes through `sources/cache.py`, and `DEMO_MODE` is respected.
9. **Check before a PR:** `pytest` for the backend, `npm run build` for the frontend. B's calc functions stay pure, with offline tests. Tests marked `@stub` expect `NotImplementedError`; remove the marker once the function works.
10. **Milestone gate:** no `stretch` issues until M4 is done, and nothing new after the H20 freeze.

Repo setup (done Oct 23)

The one-time setup is complete:

- Scaffold on `main`: FastAPI skeleton serving the mock, Vite frontend with react-leaflet and Recharts, stub modules with the signatures above, stub tests, `stations.json`, `.env.example`, `CLAUDE.md`.
- Public repo with squash-only merges, branch auto-delete, and the `main` ruleset described in Roles. CI passes on `main`.
- Labels `role:data-api`, `role:science`, `role:frontend`, `stretch`, `contract`, `plan`, `bug`; milestones M0–M6; 32 unassigned issues.

Roles: A is @Chuckisawsome12345, B is @karthikmaheshwaran, C is @Zooomer69. Each role's issues (plus #1 and #32 for everyone) are assigned, and `.github/CODEOWNERS` maps each role's folders to its owner.

Sources

Pages opened while writing this plan, plus two items still to confirm.

- CIMIS station list API: station coordinates, active flags, and Durham's 2025-07-30 disconnect date.
- Open-Meteo archive API: tested with hourly °F at Durham.
- FNRIC almond hull split model development: GDD90, single-sine method, 12 cultivars. Regression coefficients are in Tombesi et al. 2010 (not yet opened).
- **To confirm:** FNRIC's chill windows (portions Sep 1, chill hours Nov 1) came from a search summary because the chilling models page returned 403. B confirms in issue #13.
- **To confirm:** Open-Meteo's free-tier limits and how long requests are counted, on their terms page.
- **To find:** published chill-portion requirements per variety (UC sources) for the "% of chill requirement met" feature.